

Fase 0: Fundamentos, Infraestructura y Modelado de Datos (Semanas 1-2)

El objetivo de esta fase es establecer las bases que eviten refactorizaciones costosas más adelante. Un error en el modelado relacional acá paraliza la Fase 3.

- **Modelado de Base de Datos (PostgreSQL):**
 - Diseñar esquema estricto usando UUIDs v4 como Claves Primarias (PK) para **todas** las tablas centrales (Usuarios, Rutinas, Entrenamientos, Ejercicios).
 - Estructurar el patrón de inmutabilidad: La tabla **rutinas** debe tener campos **version_id** y **is_active**.
 - Configurar Alembic (o herramienta equivalente) para el control de versiones de las migraciones SQL.
- **Setup de Almacenamiento Externo (Cloudflare R2):**
 - Crear bucket en R2.
 - Implementar en FastAPI un servicio de presigned URLs usando **boto3**. El frontend subirá los archivos directamente a R2, evitando que el backend procese binarios pesados.
- **Setup Base PWA:**
 - Configurar Vite/React con el **manifest.json** y un Service Worker inicial (Workbox) para cacheo de assets estáticos (HTML, JS, CSS).

Fase 1: Core de Gestión del Entrenador (Semanas 3-4)

Se implementa el Módulo A completo, enfocado en operaciones CRUD estándar con conexión constante.

- **Autenticación y Seguridad:**
 - Implementar JWT en FastAPI.
 - Endpoints de registro y login. Hash de contraseñas con bcrypt.
- **Gestión de Perfil y Alumnos:**
 - CRUD de Perfil Profesional (Subida de foto vía R2).
 - CRUD de Alumnos (Alta, baja, estado).
 - Generación de links/códigos únicos de invitación para vincular el registro de un alumno a un entrenador específico.
- **Catálogo Maestro de Ejercicios:**
 - Poblar base de datos con ejercicios pre-cargados (SVGs alojados en R2).
 - Endpoints para que el entrenador cree ejercicios custom, subiendo multimedia a R2 y guardando la URL en PostgreSQL.

Fase 2: Motor Inmutable de Rutinas (Semanas 5-6)

Aquí se materializa la lógica de versionado. Es la fase de mayor complejidad lógica en el backend.

- **Estructura de Datos Jerárquica:**
 - Implementar endpoints transaccionales para crear la estructura completa de un solo golpe: Rutina -> Días -> Ejercicios -> Series.
- **Lógica de Versionado (Update = Insert):**
 - Desarrollar el endpoint de edición de rutinas. En lugar de actualizar registros, debe marcar la rutina actual como `is_active = false` y crear una nueva rama de registros con nuevos UUIDs, clonando los datos e insertando las modificaciones.
- **Asignación:**
 - Endpoint para vincular el UUID de una rutina activa con el UUID de un alumno.

Fase 3: Experiencia del Alumno y Motor Offline (Semanas 7-9)

El núcleo de riesgo técnico. La plataforma pasa de ser un CRUD a una aplicación robusta contra caídas de red.

- **Setup de Estado Local (TanStack Query + IndexedDB):**
 - Configurar el `PersistQueryClientProvider` en React para que las queries cacheadas (la rutina activa del alumno) se guarden en IndexedDB de forma persistente.
- **Modo Entrenamiento (Lectura Offline):**
 - Desarrollar la UI mobile-first para el registro de entrenamientos (inputs de peso, repeticiones, RPE). Esta vista solo lee del caché local.
- **Cola de Mutaciones (Escritura Offline):**
 - Implementar generación de UUIDs v4 en el frontend para cada serie/entrenamiento registrado.
 - Configurar la cola: Si la petición POST falla por falta de red, la carga útil se guarda en localforage/IndexedDB.
- **Sincronización:**
 - Lógica de "reflujo": Al detectar el evento `online` del navegador o al volver a abrir la PWA, vaciar la cola enviando los datos a FastAPI.
 - El backend debe aceptar entrenamientos que apunten a UUIDs de rutinas que estén marcadas como inactivas (resolución de inmutabilidad).

Fase 4: Agregación de Datos y Analytics (Semanas 10-11)

El valor agregado del producto. Requiere optimización a nivel SQL, no procesamiento en memoria con Python.

- **Cálculo de Récores (PRs):**
 - Implementar queries SQL o Vistas Materializadas en PostgreSQL para calcular Mejor Peso y Mejor Volumen Histórico por ejercicio y alumno.
- **Dashboards:**
 - Endpoints de agregación para el entrenador: Cálculo de adherencia, alumnos inactivos, volumen semanal.
 - Endpoints para gráficos del alumno: Evolución de cargas y días entrenados.

Fase 4.5 - Motor de Gamificación B2B y Auditoría (Semanas 11-12)

Esta fase se inserta asumiendo que el core offline (Fase 3) y el cálculo de PRs (Fase 4) están funcionales.

1. Modificación de Esquemas Relacionales (PostgreSQL)

- **Tabla **alumnos**:** Agregar campo **fecha_ultimo_peso** (DateTime).
- **Tabla estática **gamificacion_umbrales**:** (id_ejercicio, nivel_nombre, subnivel, multiplicador_requerido). 15 filas por cada uno de los 5 ejercicios habilitados (Press Banca, Sentadilla, Peso Muerto, Press Militar, Dominadas).
- **Tabla transaccional **log_ligas_alumnos**:** (id_alumno, id_ejercicio, nivel_alcanzado, fecha_logro, estado_validacion). Los estados posibles son: **aprobado_automático**, **pendiente_auditoria**, **aprobado_manual**, **rechazado**.

2. Guardián de Actualización de Peso (Frontend / Interceptor)

- **Lógica de Bloqueo (React):** Crear un middleware o un HOC (Higher-Order Component) en la PWA. Al iniciar sesión, si **now() - fecha_ultimo_peso > 90 días**, la UI entra en un estado de bloqueo modal que impide registrar nuevos entrenamientos hasta que el alumno ingrese su peso actual.
- **Endpoint de Recálculo (FastAPI):** Cuando el alumno actualiza su peso a los 90 días, el backend debe disparar un *background task* que recalcule su coeficiente de Fuerza Relativa ($\$e1RM_actual / nuevo_peso_corporal\$$) para los 5 ejercicios y actualice sus rangos, degradando o subiendo su nivel según corresponda.

3. Motor de Cálculo y Sistema Antifraude (Backend FastAPI)

- **Cálculo e1RM:** Al procesar la cola de mutaciones offline, si el ejercicio es uno de los 5 principales, FastAPI aplica la fórmula de Epley: $\$(Peso\ Levantado) \times (1 + 0.0333 \times Reps)\$$. Para dominadas se utiliza: $\$(Peso\ Corporal\ Actual + Lastre) \times (1 + 0.0333 \times Reps)\$$.
- **Clasificación de la Mutación:**

- *Caso A (Rango Cobre a Oro + Salto lógico)*: Si el salto es menor al 10% de mejora respecto a su último PR y el nivel es inferior a Platino, se inserta en `log_ligas_alumnos` como `aprobado_automatico`.
- *Caso B (Élite o Salto Irreal)*: Si el alumno alcanza Platino/Diamante, o el salto del e1RM es mayor al 10% de golpe, se inserta como `pendiente_auditoria`.

4. Interfaz de Auditoría (Dashboard del Entrenador)

- **Panel de Red Flags**: Una vista exclusiva donde el coach ve una lista de récords sospechosos o de nivel elite.
- **Métricas de contexto**: Para que el coach decida, la UI debe mostrarle el récord anterior vs. el récord reclamado.
- **Acciones transaccionales**: Botones de "Aprobar" (actualiza el estado y notifica) o "Rechazar" (elimina el registro falso para no ensuciar los gráficos de progreso).

5. Interfaz de Ligas y Notificaciones (Experiencia del Alumno)

- **Sección "Mi Liga" (PWA)**: Componente visual con los 5 ejercicios, mostrando la insignia actual (SVGs desde Cloudflare R2) y una barra de progreso que indique cuánto peso le falta en su e1RM para el próximo subnivel (basado en su peso corporal actual).
- **Manejo de UI Optimista**: Si el récord pasa a `pendiente_auditoria`, la insignia en la PWA debe aparecer bloqueada o en marca de agua con un badge: *"En revisión por tu entrenador"*.
- **Notificación asíncrona**: Integración de un servicio de mailing (ej. Resend) en FastAPI para enviar el correo al alumno únicamente cuando un récord en auditoría es aprobado manualmente.

Fase 5: Estabilización, Testing y Despliegue (Semana 12)

- Pruebas de estrés de la lógica offline: Simular cortes de red en medio de un entrenamiento y forzar cierres de la app.
- Auditoría de índices en PostgreSQL para asegurar tiempos de respuesta menores a 2 segundos en el dashboard.
- Deploy del backend en infraestructura cloud y del frontend estático